

1. Install git.

Objective

To install Git on an Amazon Linux EC2 server and configure it so that Git can be used for version control and managing project files.

Purpose

Git is a distributed version control system used to track changes in files, manage different versions of a project, create branches, commit changes, and collaborate with remote repositories such as GitHub.

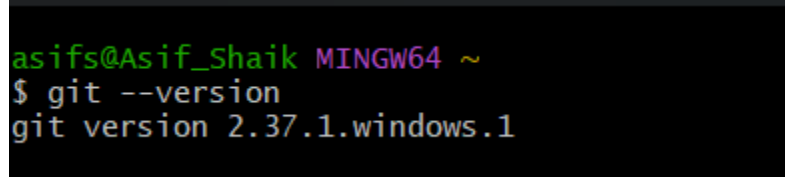
Implementation

Step 1: Install Git

```
dnf install git -y
```



```
git --version
```



```
asifs@Asif_Shaik MINGW64 ~  
$ git --version  
git version 2.37.1.windows.1
```

Conclusion

Git was successfully installed and configured on the EC2 server. Git can now be used to create repositories, track changes, create branches, commit changes, and push or pull code from GitHub.

2. Create a repo in github with README.md and .ignore file.

Objective

To create a new repository in GitHub with a README.md file and a .gitignore file for managing and documenting the project.

Purpose

- GitHub Repository: Stores and manages project files remotely.
- README.md: Provides information and documentation about the project.
- .gitignore: Specifies files that Git should not track or upload to the repository.
- Helps maintain a clean and organized project.

Implementation

Step 1: Open GitHub

Go to [GitHub](#) and log in.

Step 2: Create a new repository

Click:

+ → New repository

Enter a repository name, for example:

devops-git-practice

Step 3: Add README.md

Select:

☒ Add a README file

This automatically creates README.md.

Step 4: Create the repository

Click:

Create repository

Step 5: Create .gitignore

Inside the repository, click:

Add file → Create new file

Enter the filename:

.gitignore

Add entries such as:

*.log

*.tmp

.env

Then click:

Commit changes

Step 6: Verify the repository

The repository should contain:

devops-git-practice/

├── .gitignore

└── README.md

```

MINGW64/c/Users/asifs/devops-git-practice
asifs@Asif_Shaik MINGW64 ~
$ git clone https://github.com/asifshaik6558/devops-git-practice.git
Cloning into 'devops-git-practice'...
warning: You appear to have cloned an empty repository.

asifs@Asif_Shaik MINGW64 ~
$ cd devops-git-practice

asifs@Asif_Shaik MINGW64 ~/devops-git-practice (main)
$ echo "# devops-git-practice" > README.md

asifs@Asif_Shaik MINGW64 ~/devops-git-practice (main)
$ touch .gitignore

asifs@Asif_Shaik MINGW64 ~/devops-git-practice (main)
$ cat > .gitignore <<EOF
> *.log
> *.tmp
> .env
> EOF

asifs@Asif_Shaik MINGW64 ~/devops-git-practice (main)
$ ls -la
total 22
drwxr-xr-x 1 asifs 197609  0 Aug 20 14:47 ./
drwxr-xr-x 1 asifs 197609  0 Aug 20 14:47 ../
drwxr-xr-x 1 asifs 197609  0 Aug 20 14:47 .git/
-rw-r--r-- 1 asifs 197609 17 Aug 20 14:48 .gitignore
-rw-r--r-- 1 asifs 197609 22 Aug 20 14:47 README.md

asifs@Asif_Shaik MINGW64 ~/devops-git-practice (main)
$ git add README.md .gitignore
warning: in the working copy of '.gitignore', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'README.md', LF will be replaced by CRLF the next time Git touches it

asifs@Asif_Shaik MINGW64 ~/devops-git-practice (main)
$ git commit -m "Add README and gitignore"
[main (root-commit) 0ffbfbf4] Add README and gitignore
 2 files changed, 4 insertions(+)
 create mode 100644 .gitignore
 create mode 100644 README.md

asifs@Asif_Shaik MINGW64 ~/devops-git-practice (main)
$ AC

asifs@Asif_Shaik MINGW64 ~/devops-git-practice (main)
$ AC

asifs@Asif_Shaik MINGW64 ~/devops-git-practice (main)
$ git push -u origin main
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 12 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (4/4), 309 bytes | 309.00 KiB/s, done.
Total 4 (delta 0), reused 0 (delta 0), pack-reused 0
To https://github.com/asifshaik6558/devops-git-practice.git

```

The screenshot shows the GitHub web interface for a repository named 'devops-git-practice' owned by 'asifshaik6558'. The repository is public and has 1 branch (main) and 0 tags. The commit history shows a single commit '0ffbfbf4' from 2 minutes ago, titled 'Add README and gitignore', which added the files '.gitignore' and 'README.md'. The README file is visible and contains the text 'devops-git-practice'. On the right sidebar, the 'About' section describes it as 'Git and DevOps practice'. The 'Releases' and 'Packages' sections indicate no releases or packages have been published yet. The 'Contributors' section lists 'asifshaik6558' as the sole contributor.

Conclusion

A GitHub repository was successfully created with README.md and .gitignore. The README.md provides project documentation, while .gitignore prevents unwanted files from being tracked by Git.

3. Clone the created repo to local.

Objective

To clone the repository created on **GitHub** to the **local system** so that the project files can be accessed, modified, and managed using Git.

Purpose

Cloning creates a **local copy of the remote GitHub repository**. It allows developers to work on the project locally and later synchronize changes with GitHub using git push and git pull.

Implementation / Steps

Step 1: Go to the local directory

Open Git Bash and run:

```
cd ~
```

Step 2: Clone the GitHub repository

```
git clone https://github.com/asifshaik6558/devops-git-practice.git
```

Step 3: Enter the cloned repository

```
cd devops-git-practice
```

Step 4: Check the files

```
ls -la
```

Step 5: Verify the remote repository

```
git remote -v
```

```

asifs@Asif_Shaik MINGW64 ~/devops-git-practice (main)
$ git clone https://github.com/asifshaik6558/devops-git-practice.git
Cloning into 'devops-git-practice'...
remote: Enumerating objects: 4, done.
remote: Counting objects: 100% (4/4), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 4 (delta 0), reused 4 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (4/4), done.

asifs@Asif_Shaik MINGW64 ~/devops-git-practice (main)
$ cd devops-git-practice

asifs@Asif_Shaik MINGW64 ~/devops-git-practice/devops-git-practice (main)
$ ls -la
total 10
drwxr-xr-x 1 asifs 197609  0 Aug 20 14:55 ./
drwxr-xr-x 1 asifs 197609  0 Aug 20 14:55 ../
drwxr-xr-x 1 asifs 197609  0 Aug 20 14:55 .git/
-rw-r--r-- 1 asifs 197609 20 Aug 20 14:55 .gitignore
-rw-r--r-- 1 asifs 197609 23 Aug 20 14:55 README.md

asifs@Asif_Shaik MINGW64 ~/devops-git-practice/devops-git-practice (main)
$ git remote -v
origin https://github.com/asifshaik6558/devops-git-practice.git (fetch)
origin https://github.com/asifshaik6558/devops-git-practice.git (push)

```

4. Create two files in local repo.

Objective

To create two new files inside the local Git repository and verify that Git identifies them as new, untracked files.

Purpose

Creating files in a local repository is useful for adding new project content and practicing Git operations such as git add, git commit, and git push.

Implementation / Steps

Step 1: Go to the local repository

```
cd ~/devops-git-practice
```

Step 2: Create two files

```
touch file1.txt file2.txt
```

Step 3: Verify the files

ls

You should see:

README.md

file1.txt

file2.txt

Step 4: Check Git status

git status

Git should show:

Untracked files:

file1.txt

file2.txt

```
asifs@Asif_Shaik MINGW64 ~/devops-git-practice (main)
$ ls
README.md  devops-git-practice/

asifs@Asif_Shaik MINGW64 ~/devops-git-practice (main)
$ ls devops-git-practice
README.md

asifs@Asif_Shaik MINGW64 ~/devops-git-practice (main)
$ touch file1.txt file2.txt

asifs@Asif_Shaik MINGW64 ~/devops-git-practice (main)
$ ls
README.md  devops-git-practice/  file1.txt  file2.txt

asifs@Asif_Shaik MINGW64 ~/devops-git-practice (main)
$ git status
On branch main
Your branch is up to date with 'origin/main'.

Untracked files:
  (use "git add <file>..." to include in what will be committed)
      devops-git-practice/
      file1.txt
      file2.txt

nothing added to commit but untracked files present (use "git add" to track)
asifs@Asif_Shaik MINGW64 ~/devops-git-practice (main)
```

Conclusion

Two files, file1.txt and file2.txt, were successfully created in the local repository. Git recognizes them as untracked files, ready to be added, committed, and pushed to the central GitHub repository.

5. Commit two files and push to central Repository.

Objective

To save the changes made to file1.txt and file2.txt in Git history and upload the committed changes from the local repository to the central GitHub repository.

Purpose

- git add → Moves files to the staging area.
- git commit → Permanently records the changes in the local Git repository.
- git push → Uploads the committed changes to the central GitHub repository.
- This keeps the local and remote repositories synchronized.

Implementation / Steps

Step 1: Go to the local repository

```
cd ~/devops-git-practice
```

Step 2: Add the two files

```
git add file1.txt file2.txt
```

Step 3: Check the staged files

```
git status
```

You should see:

Changes to be committed:

new file: file1.txt

new file: file2.txt

Step 4: Commit the files

```
git commit -m "Add two files"
```

Step 5: Push to the central repository

git push origin main

Here:

- origin = GitHub remote repository
- main = branch being pushed

Step 6: Verify

git status

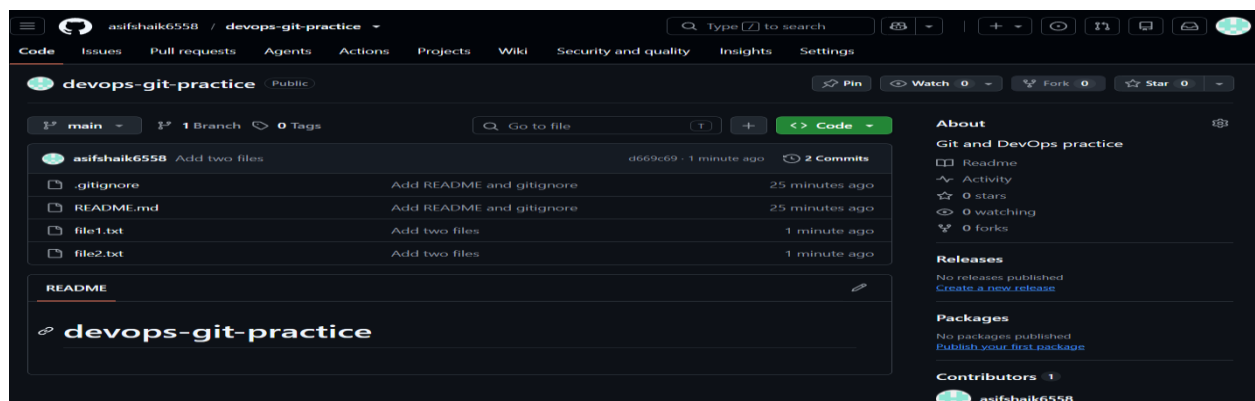
```
asifs@Asif_Shaik MINGW64 ~/devops-git-practice (main)
$ git add file1.txt file2.txt

asifs@Asif_Shaik MINGW64 ~/devops-git-practice (main)
$ git commit -m "Add two files"
[main d669c69] Add two files
 2 files changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 file1.txt
 create mode 100644 file2.txt

asifs@Asif_Shaik MINGW64 ~/devops-git-practice (main)
$ git push origin main
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 12 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 316 bytes | 316.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0
To https://github.com/asifshaik6558/devops-git-practice.git
   Offbbbf4..d669c69  main -> main

asifs@Asif_Shaik MINGW64 ~/devops-git-practice (main)
$ git status
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
```



Conclusion

The two files, file1.txt and file2.txt, were successfully added and committed to the local Git repository and pushed to the central GitHub repository. This demonstrates the basic Git workflow of add → commit → push.

6. Create a branch in local and create a sample file and push to central.

Objective

To create a new branch in the **local Git repository**, create a sample file in that branch, commit the file, and push the branch to the **central GitHub repository**.

Purpose

- **Branch** → Provides a separate workspace for development.
- **Sample file** → Demonstrates adding new content to the branch.
- **Commit** → Saves the changes in Git history.
- **Push** → Uploads the branch and its changes to GitHub.
- This allows development without directly modifying the main branch.

Implementation / Steps

Step 1: Go to the local repository

```
cd ~/devops-git-practice
```

Step 2: Create a new branch

```
git switch -c feature-branch
```

Verify:

```
git branch
```

Expected:

```
* feature-branch  
main
```

Step 3: Create a sample file

```
touch sample.txt
```

Verify:

ls

You should see:

README.md

file1.txt

file2.txt

sample.txt

Step 4: Add the sample file

git add sample.txt

Step 5: Commit the file

git commit -m "Add sample file"

Step 6: Push the branch to the central repository

git push -u origin feature-branch

Here:

- origin → GitHub central repository
- feature-branch → branch being pushed
- -u → connects the local branch with the remote branch

Step 7: Verify

git branch -a

```

asifs@Asif_Shaik MINGW64 ~/devops-git-practice (main)
$ git branch feature

asifs@Asif_Shaik MINGW64 ~/devops-git-practice (main)
$ git switch feature
Switched to branch 'feature'

asifs@Asif_Shaik MINGW64 ~/devops-git-practice (feature)
$ git branch
* feature
  main

asifs@Asif_Shaik MINGW64 ~/devops-git-practice (feature)
$ touch sample.txt

asifs@Asif_Shaik MINGW64 ~/devops-git-practice (feature)
$ ls
README.md  file1.txt  file2.txt  sample.txt

asifs@Asif_Shaik MINGW64 ~/devops-git-practice (feature)
$ git add sample.txt

asifs@Asif_Shaik MINGW64 ~/devops-git-practice (feature)
$ git commit -m "Add sample file"
[feature 584debc] Add sample file
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 sample.txt

asifs@Asif_Shaik MINGW64 ~/devops-git-practice (feature)
$ git push -u origin feature
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Delta compression using up to 12 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (2/2), 233 bytes | 233.00 KiB/s, done.
Total 2 (delta 1), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
remote:
remote: Create a pull request for 'feature' on GitHub by visiting:
remote:   https://github.com/asifshaik6558/devops-git-practice/pull/new/feature
remote:
To https://github.com/asifshaik6558/devops-git-practice.git
 * [new branch]      feature -> feature
branch 'feature' set up to track 'origin/feature'.

asifs@Asif_Shaik MINGW64 ~/devops-git-practice (feature)
$ git branch -a
* feature
  main
remotes/origin/feature
remotes/origin/main

```

The screenshot shows the GitHub interface for the repository 'devops-git-practice'. The 'feature' branch is selected, which is 1 commit ahead of 'main'. A recent commit by 'asifshaik6558' is shown with the message 'Add sample file'. Below the commit, a list of files is displayed, including '.gitignore', 'README.md', 'file1.txt', 'file2.txt', and 'sample.txt'. The right sidebar contains sections for 'About', 'Releases', 'Packages', and 'Contributors'.

Conclusion

A new feature-branch was created locally, a sample.txt file was created and committed, and the branch was successfully pushed to the central GitHub repository. This demonstrates the Git workflow of branch → create → add → commit → push.

7. Create a branch in github and clone that to local.

Objective

To create a new branch in the **GitHub remote repository** and clone that specific branch to the **local system** for development and testing.

Purpose

The purpose is to work on a separate branch without directly modifying the `main` branch. This allows developers to safely make and test changes before merging them into the main project.

Implementation

Step 1: Open the GitHub repository

Open your repository:

[devops-git-practice on GitHub](#)

Step 2: Create a branch in GitHub

On the repository page:

1. Click the branch selector.
2. Enter a new branch name, for example:
`github-branch`
3. Click Create branch.

The branch is now available in GitHub.

Step 3: Go to the local home directory

```
cd ~
```

Step 4: Clone the specific GitHub branch

```
git clone -b github-branch https://github.com/asifshaik6558/devops-git-practice.git github-branch-local
```

Here:

- `-b github-branch` → clones the specified branch.
- <https://github.com/...> → GitHub repository.
- `github-branch-local` → local directory name.

Step 5: Enter the cloned repository

```
cd github-branch-local
```

Step 6: Verify the branch

```
git branch
```

Expected:

```
* github-branch
```

Step 7: Verify the remote repository

```
git remote -v
```

```
asifs@Asif_Shaik MINGW64 ~/devops-git-practice (feature)
$ cd ~

asifs@Asif_Shaik MINGW64 ~
$ git clone -b github-branch https://github.com/asifshaik6558/devops-git-practice.git github-branch-local
Cloning into 'github-branch-local'...
remote: Enumerating objects: 9, done.
remote: Counting objects: 100% (9/9), done.
remote: Compressing objects: 100% (5/5), done.
remote: Total 9 (delta 2), reused 8 (delta 1), pack-reused 0 (from 0)
Receiving objects: 100% (9/9), done.
Resolving deltas: 100% (2/2), done.

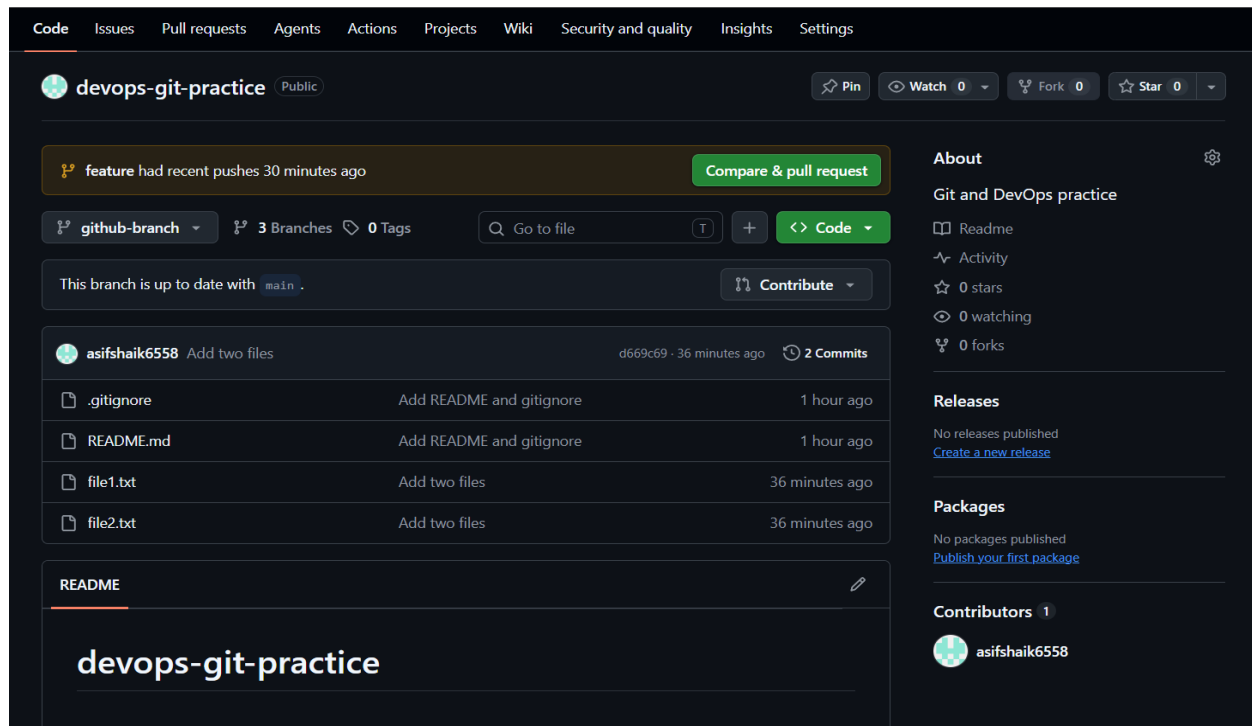
asifs@Asif_Shaik MINGW64 ~
$ git branch
fatal: not a git repository (or any of the parent directories): .git

asifs@Asif_Shaik MINGW64 ~
$ cd github-branch-local

asifs@Asif_Shaik MINGW64 ~/github-branch-local (github-branch)
$ git branch
* github-branch

asifs@Asif_Shaik MINGW64 ~/github-branch-local (github-branch)
$ ls -la
total 26
drwxr-xr-x 1 asifs 197609  0 Aug 20 15:46 ./
drwxr-xr-x 1 asifs 197609  0 Aug 20 15:48 ../
drwxr-xr-x 1 asifs 197609  0 Aug 20 15:46 .git/
-rw-r--r-- 1 asifs 197609 20 Aug 20 15:46 .gitignore
-rw-r--r-- 1 asifs 197609 23 Aug 20 15:46 README.md
-rw-r--r-- 1 asifs 197609  0 Aug 20 15:46 file1.txt
-rw-r--r-- 1 asifs 197609  0 Aug 20 15:46 file2.txt

asifs@Asif_Shaik MINGW64 ~/github-branch-local (github-branch)
$ git remote -v
origin https://github.com/asifshaik6558/devops-git-practice.git (fetch)
origin https://github.com/asifshaik6558/devops-git-practice.git (push)
```



Conclusion

A new branch was successfully created in GitHub and cloned to the local system. The local repository is now connected to the github-branch, allowing changes to be developed and tested separately from the main branch.

8. Merge the created branch with master in git local.

Objective

To merge the changes from the created **github-branch** into the **master** branch in the local Git repository.

Purpose

The purpose of merging is to **combine the work completed in one branch with another branch**. This allows the changes developed in github-branch to become part of the master branch.

Implementation

Step 1: Go to the local repository

```
cd ~/github-branch-local
```

Step 2: Check available branches

```
git branch
```

Step 3: Switch to the master branch

If master exists:

```
git switch master
```

Step 4: Merge the created branch

```
git merge github-branch
```

If the merge is successful, you may see:

Merge made by the 'ort' strategy.

Step 5: Check the status

```
git status
```

You should see something like:

On branch master

nothing to commit, working tree clean

If your repository uses main instead of master

Your previous repository used main, so if master does not exist, use:

```
git switch main
```

```
git merge github-branch
```

```
git status
```

If you see:

Already up to date.

```
asifs@Asif_Shaik MINGW64 ~/github-branch-local (github-branch)
$ git branch
* github-branch

asifs@Asif_Shaik MINGW64 ~/github-branch-local (github-branch)
$ git switch main
Switched to a new branch 'main'
branch 'main' set up to track 'origin/main'.

asifs@Asif_Shaik MINGW64 ~/github-branch-local (main)
$ git merge github-branch
Already up to date.

asifs@Asif_Shaik MINGW64 ~/github-branch-local (main)
$ git status
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
```

Conclusion

The changes from github-branch were merged into the master branch locally. This demonstrates how Git branches can be used for separate development and then combined into the main development branch.

9. Merge the created branch with master in github by sending a pull request.

Objective

To merge the changes from the created github-branch into the master branch in GitHub by creating and completing a Pull Request (PR).

Purpose

A Pull Request is used to propose changes from one branch to another. It allows the changes to be reviewed before they are merged into the main branch.

In simple terms:

github-branch
↓
Pull Request

↓
Review
↓
Merge
↓
master

Implementation / Steps

Step 1: Push your branch to GitHub

From your local repository:

```
git push -u origin github-branch
```

Step 2: Open your GitHub repository

Open:

[devops-git-practice on GitHub](#)

Step 3: Select the branch

Click the branch selector and select:

github-branch

Step 4: Create a Pull Request

GitHub may display:

Compare & pull request

Click it.

If it does not appear:

1. Go to Pull requests
2. Click New pull request
3. Select:

base: master

compare: github-branch

If your repository uses main instead of master, select main as the base branch.

Step 5: Add Pull Request details

Enter a title such as:

Merge github-branch into master

Add a description if required.

Then click:

Create pull request

Step 6: Review the Pull Request

Check the changes shown under the Files changed section.

If everything is correct, click:

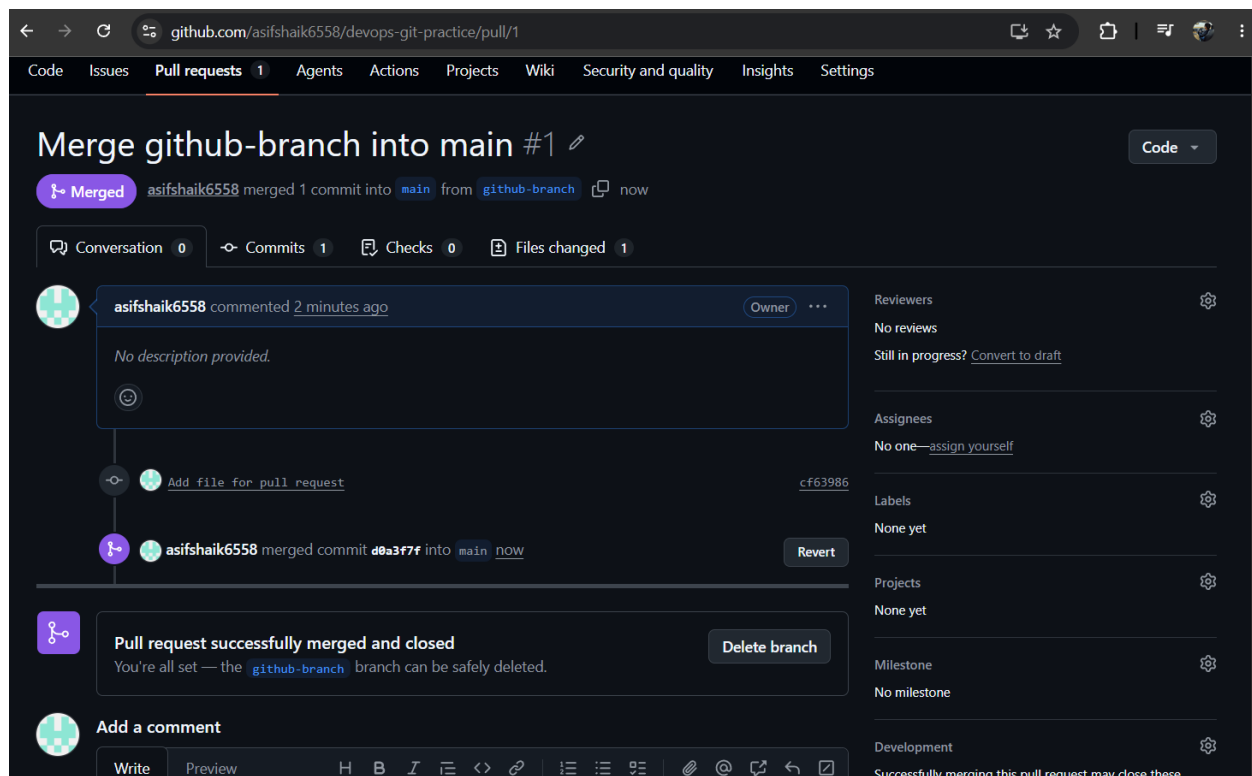
Merge pull request

Then click:

Confirm merge

Step 7: Verify the merge

Go to the Code tab and select the master branch.



Conclusion

The changes from github-branch were submitted to GitHub through a Pull Request and merged into the master branch after review. This is the standard GitHub workflow for safely reviewing and integrating changes into the main branch.

10. Create a file in local and send that to branch in github.

Objective

To create a new file in the local Git repository, commit the file, and push it to the github-branch branch in GitHub.

Purpose

The purpose is to practice transferring changes from a local repository to a specific remote branch using the Git workflow:

Create file → Add → Commit → Push → GitHub branch

Implementation / Steps

Step 1: Go to your local repository

```
cd ~/github-branch-local
```

Step 2: Switch to github-branch

```
git switch github-branch
```

Verify:

```
git branch
```

You should see:

```
* github-branch  
main
```

Step 3: Create a new file

For example:

```
touch local-file.txt
```

Check:

```
ls
```

Step 4: Check Git status

```
git status
```

You should see:

```
Untracked files:  
  local-file.txt
```

Step 5: Add the file

```
git add local-file.txt
```

Step 6: Commit the file

```
git commit -m "Add local file"
```

Step 7: Push the file to the GitHub branch

```
git push origin github-branch
```

If this is the first time pushing this local branch:

```
git push -u origin github-branch
```

Step 8: Verify on GitHub

Open your repository and select:

github-branch

```
asifs@Asif_Shaik MINGW64 ~/github-branch-local (github-branch)
$ cd ~/github-branch-local

asifs@Asif_Shaik MINGW64 ~/github-branch-local (github-branch)
$ git switch github-branch
Already on 'github-branch'
Your branch is up to date with 'origin/github-branch'.

asifs@Asif_Shaik MINGW64 ~/github-branch-local (github-branch)
$ git branch
* github-branch
  main

asifs@Asif_Shaik MINGW64 ~/github-branch-local (github-branch)
$ touch local-file.txt

asifs@Asif_Shaik MINGW64 ~/github-branch-local (github-branch)
$ ls
README.md file1.txt file2.txt local-file.txt pr-test.txt

asifs@Asif_Shaik MINGW64 ~/github-branch-local (github-branch)
$ git status
On branch github-branch
Your branch is up to date with 'origin/github-branch'.

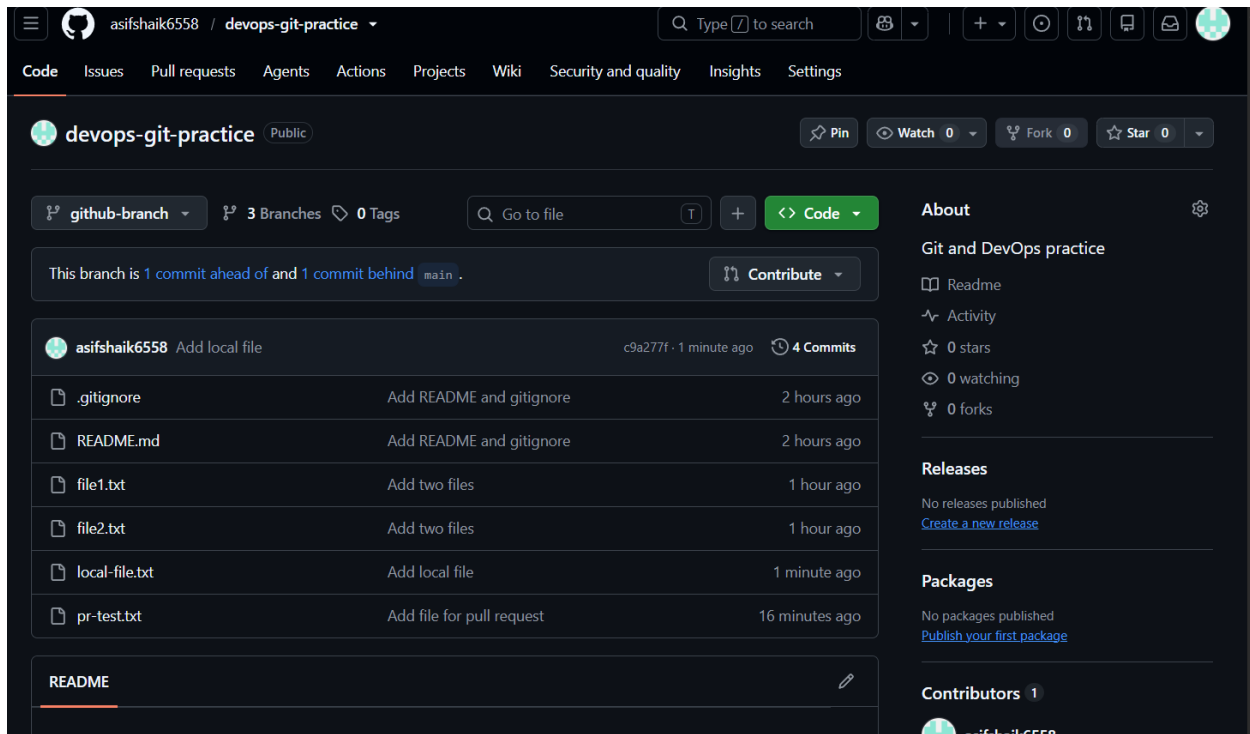
Untracked files:
  (use "git add <file>..." to include in what will be committed)
    local-file.txt

nothing added to commit but untracked files present (use "git add" to track)

asifs@Asif_Shaik MINGW64 ~/github-branch-local (github-branch)
$ git add local-file.txt

asifs@Asif_Shaik MINGW64 ~/github-branch-local (github-branch)
$ git commit -m "Add local file"
[github-branch c9a277f] Add local file
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 local-file.txt

asifs@Asif_Shaik MINGW64 ~/github-branch-local (github-branch)
$ git push origin github-branch
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Delta compression using up to 12 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (2/2), 234 bytes | 234.00 KiB/s, done.
Total 2 (delta 1), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/asifshaik6558/devops-git-practice.git
   cf63986..c9a277f github-branch -> github-branch
```



Conclusion

A new file was created in the local repository, committed to Git, and pushed successfully to the github-branch on GitHub. This demonstrates how local changes can be sent to a specific remote branch using `git add` → `git commit` → `git push`.

11. Clone only a branch from github to local.

Objective

To clone only a specific branch from a GitHub repository to the local system instead of cloning the default main branch.

Purpose

Cloning a specific branch is useful when you want to work directly with a particular feature or development branch without initially checking out the main branch.

Implementation / Steps

Step 1: Go to your local home directory

```
cd ~
```

Step 2: Clone only the required branch

For your github-branch:

```
git clone -b github-branch --single-branch https://github.com/asifshaik6558/devops-git-practice.git github-branch-only
```

Here:

- -b github-branch → selects the branch to clone.
- --single-branch → downloads only that branch.
- github-branch-only → creates the local directory.

Step 3: Enter the cloned repository

```
cd github-branch-only
```

Step 4: Verify the current branch

```
git branch
```

Expected:

```
* github-branch
```

Step 5: Verify the remote

```
git remote -v
```

Step 6: Check the files

```
ls -la
```

```

asifs@Asif_Shaik MINGW64 ~/github-branch-local (github-branch)
$ cd ~

asifs@Asif_Shaik MINGW64 ~
$ git clone -b github-branch --single-branch https://github.com/asifshaik6558/devops-git-practice.git github-branch-only
Cloning into 'github-branch-only'...
remote: Enumerating objects: 11, done.
remote: Counting objects: 100% (11/11), done.
remote: Compressing objects: 100% (7/7), done.
remote: Total 11 (delta 3), reused 9 (delta 1), pack-reused 0 (from 0)
Receiving objects: 100% (11/11), done.
Resolving deltas: 100% (3/3), done.

asifs@Asif_Shaik MINGW64 ~
$ cd github-branch-only

asifs@Asif_Shaik MINGW64 ~/github-branch-only (github-branch)
$ git branch
* github-branch

asifs@Asif_Shaik MINGW64 ~/github-branch-only (github-branch)
$ git remote -v
origin https://github.com/asifshaik6558/devops-git-practice.git (fetch)
origin https://github.com/asifshaik6558/devops-git-practice.git (push)

asifs@Asif_Shaik MINGW64 ~/github-branch-only (github-branch)
$ ls -la
total 26
drwxr-xr-x 1 asifs 197609  0 Aug 20 17:07 ./
drwxr-xr-x 1 asifs 197609  0 Aug 20 17:07 ../
drwxr-xr-x 1 asifs 197609  0 Aug 20 17:07 .git/
-rw-r--r-- 1 asifs 197609 20 Aug 20 17:07 .gitignore
-rw-r--r-- 1 asifs 197609 23 Aug 20 17:07 README.md
-rw-r--r-- 1 asifs 197609  0 Aug 20 17:07 file1.txt
-rw-r--r-- 1 asifs 197609  0 Aug 20 17:07 file2.txt
-rw-r--r-- 1 asifs 197609  0 Aug 20 17:07 local-file.txt
-rw-r--r-- 1 asifs 197609  0 Aug 20 17:07 pr-test.txt

```

Conclusion

The github-branch was successfully cloned from GitHub as a single branch into the local system. Using --single-branch ensures that Git initially fetches only the specified branch, making it useful when you need to work with a particular branch.

12.Create a file with all passwords and make that untrackable with git.

Objective

To create a password file locally and configure Git to ignore the file, so it is not tracked or pushed to GitHub.

Purpose

.gitignore tells Git which files should not be tracked. This is important for protecting sensitive files such as passwords, credentials, API keys, and environment files.

Implementation / Steps

1. Go to your local repository

```
cd ~/devops-git-practice
```

2. Create a password file

`touch passwords.txt`

You can add dummy data:

```
echo "username=demo" > passwords.txt
echo "password=dummy123" >> passwords.txt
```

3. Add the file to .gitignore

Open:

`vi .gitignore`

Add:

`passwords.txt`

Save and exit:

Esc

`:wq`

4. Check Git status

`git status`

You should not see `passwords.txt` under untracked files.

5. Verify that Git is ignoring the file

`git check-ignore -v passwords.txt`

You should get output similar to:

```
.gitignore:1:passwords.txt passwords.txt
```

6. Check the file exists locally

`ls -la`

```

asifs@Asif_Shaik MINGW64 ~/github-branch-only (github-branch)
$ cd ~/devops-git-practice

asifs@Asif_Shaik MINGW64 ~/devops-git-practice (feature)
$ touch passwords.txt

asifs@Asif_Shaik MINGW64 ~/devops-git-practice (feature)
$ echo "username=demo" > passwords.txt

asifs@Asif_Shaik MINGW64 ~/devops-git-practice (feature)
$ echo "password=dummy123" >> passwords.txt

asifs@Asif_Shaik MINGW64 ~/devops-git-practice (feature)
$ vi .gitignore

asifs@Asif_Shaik MINGW64 ~/devops-git-practice (feature)
$ git status
On branch feature
Your branch is up to date with 'origin/feature'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   .gitignore

no changes added to commit (use "git add" and/or "git commit -a")

asifs@Asif_Shaik MINGW64 ~/devops-git-practice (feature)
$ git check-ignore -v passwords.txt
.gitignore:4:passwords.txt      passwords.txt

```

Conclusion

A password file was created in the local repository and added to .gitignore. Git will now **ignore the password file**, preventing it from being tracked or accidentally pushed to GitHub. This helps protect sensitive information and keeps the repository secure.

13. Make a commit and make that commit reset without savings changes.

Objective

To create a Git commit and then use git reset to remove that commit while also removing the changes from the working directory.

Purpose

This demonstrates how to undo a commit completely when you no longer want the committed changes.

The command used is:

```
git reset --hard HEAD~1
```

- HEAD~1 → moves back one commit.
- --hard → removes the changes from the staging area and working directory.

Implementation / Steps

Step 1: Check your current status

```
git status
```

Step 2: Create a sample file

```
touch reset-test.txt
```

Step 3: Add the file

```
git add reset-test.txt
```

Step 4: Create a commit

```
git commit -m "Add reset test file"
```

Step 5: Check the commit

```
git log --oneline -2
```

Step 6: Reset the commit and discard its changes

```
git reset --hard HEAD~1
```

Step 7: Verify

```
git status
```

And check:

```
Ls
```

```

asifs@Asif_Shaik MINGW64 ~/devops-git-practice (feature)
$ git status
On branch feature
Your branch is up to date with 'origin/feature'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   .gitignore

no changes added to commit (use "git add" and/or "git commit -a")

asifs@Asif_Shaik MINGW64 ~/devops-git-practice (feature)
$ touch reset-test.txt

asifs@Asif_Shaik MINGW64 ~/devops-git-practice (feature)
$ git add reset-test.txt

asifs@Asif_Shaik MINGW64 ~/devops-git-practice (feature)
$ git commit -m "Add reset test file"
[feature 939c571] Add reset test file
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 reset-test.txt

asifs@Asif_Shaik MINGW64 ~/devops-git-practice (feature)
$ git log --oneline -2
939c571 (HEAD -> feature) Add reset test file
584debc (origin/feature) Add sample file

asifs@Asif_Shaik MINGW64 ~/devops-git-practice (feature)
$ git reset --hard HEAD~1
HEAD is now at 584debc Add sample file

asifs@Asif_Shaik MINGW64 ~/devops-git-practice (feature)
$ git status
On branch feature
Your branch is up to date with 'origin/feature'.

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        passwords.txt

nothing added to commit but untracked files present (use "git add" to track)

asifs@Asif_Shaik MINGW64 ~/devops-git-practice (feature)
$ ls
README.md file1.txt file2.txt passwords.txt sample.txt

```

Conclusion

A commit was successfully created and then removed using:

```
git reset --hard HEAD~1
```

The commit was removed from the local branch, and the changes introduced by that commit were also discarded. This is useful when you want to completely undo a recent local commit.

14. Revert a committed commit to the older version.

Objective

To undo the changes made by a committed commit and restore the project to the previous version while keeping the Git history safe.

Purpose

git revert is used to undo a previous commit without deleting that commit from Git history.

This is safer than git reset when the commit has already been pushed to GitHub.

Difference

Command	Purpose
git reset --hard HEAD~1	Removes the commit and changes locally
git revert <commit-id>	Creates a new commit that reverses the previous commit

Implementation / Steps

Step 1: Check commit history

```
git log --oneline
```

Example:

```
a1b2c3d Add sample file
```

```
7e8f9g Initial commit
```

Step 2: Revert the required commit

If you want to undo a1b2c3d:

```
git revert a1b2c3d
```

Git may open an editor for the revert commit message. Save and exit.

Step 3: Check the status

```
git status
```

Step 4: Check the history

`git log --oneline`

You will see something like:

```
xxxxxxx Revert "Add sample file"
a1b2c3d Add sample file
7e8f9g Initial commit
```

Notice that the original commit is still present. Git created a new revert commit that reverses its changes.

Step 5: Push the revert to GitHub

If the original commit was already pushed:

`git push origin main`

```
asifs@Asif_Shaik MINGW64 ~/devops-git-practice (feature)
$ git log --oneline
055b737 (HEAD -> feature) Revert "Add sample file"
584debc (origin/feature) Add sample file
d669c69 (origin/main, main) Add two files
0ffbbf4 Add README and gitignore

asifs@Asif_Shaik MINGW64 ~/devops-git-practice (feature)
$ git status
On branch feature
Your branch is ahead of 'origin/feature' by 1 commit.
  (use "git push" to publish your local commits)

Untracked files:
  (use "git add <file>..." to include in what will be committed)
  passwords.txt

nothing added to commit but untracked files present (use "git add" to track)

asifs@Asif_Shaik MINGW64 ~/devops-git-practice (feature)
$ ls
README.md  file1.txt  file2.txt  passwords.txt

asifs@Asif_Shaik MINGW64 ~/devops-git-practice (feature)
$ git push origin feature
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Delta compression using up to 12 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (2/2), 250 bytes | 250.00 KiB/s, done.
Total 2 (delta 1), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/asifshaik6558/devops-git-practice.git
  584debc..055b737  feature -> feature

asifs@Asif_Shaik MINGW64 ~/devops-git-practice (feature)
$ ^C

asifs@Asif_Shaik MINGW64 ~/devops-git-practice (feature)
$ git log --oneline
055b737 (HEAD -> feature, origin/feature) Revert "Add sample file"
584debc Add sample file
d669c69 (origin/main, main) Add two files
0ffbbf4 Add README and gitignore

asifs@Asif_Shaik MINGW64 ~/devops-git-practice (feature)
$ git status
On branch feature
Your branch is up to date with 'origin/feature'.

Untracked files:
  (use "git add <file>..." to include in what will be committed)
  passwords.txt

nothing added to commit but untracked files present (use "git add" to track)
asifs@Asif_Shaik MINGW64 ~/devops-git-practice (feature)
```

Conclusion

The Add sample file commit was reverted and the revert commit was successfully pushed to GitHub. The original commit remains in history, while its changes have been undone.

15.Push a file to stash without savings the changes and work on another file.

Objective

To temporarily store changes in a file using Git Stash without committing them, and then work on another file.

Purpose

git stash temporarily saves your uncommitted changes and gives you a clean working directory. This is useful when you are working on one task and suddenly need to work on another task.

Note: git stash does not push the file to GitHub. It stores the changes locally in Git's stash.

Implementation / Steps

Step 1: Go to your repository

```
cd ~/devops-git-practice
```

Step 2: Check your branch

```
git branch
```

Step 3: Create and modify the first file

```
touch file1.txt
```

Add some content:

```
echo "Work on file 1" > file1.txt
```

Check:

```
git status
```

Step 4: Add the file to Git's staging area

```
git add file1.txt
```

Step 5: Stash the changes

For staged changes:

```
git stash push -m "Work on file1"
```

If the file is untracked, use:

```
git stash push -u -m "Work on file1"
```

The -u includes untracked files.

Step 6: Check the working directory

```
git status
```

You should have a clean working tree.

Step 7: Work on another file

```
touch file2.txt
```

Add some content:

```
echo "Work on file 2" > file2.txt
```

Check:

```
git status
```

Now you can work on file2.txt while your file1.txt work is safely stored in the stash.

Step 8: View the stash

```
git stash list
```

```

asifs@Asif_Shaik MINGW64 ~/devops-git-practice (feature)
$ touch file1.txt

asifs@Asif_Shaik MINGW64 ~/devops-git-practice (feature)
$ vi file1.txt

asifs@Asif_Shaik MINGW64 ~/devops-git-practice (feature)
$ git status
On branch feature
Your branch is up to date with 'origin/feature'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
    modified:   file1.txt

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    passwords.txt

no changes added to commit (use "git add" and/or "git commit -a")

asifs@Asif_Shaik MINGW64 ~/devops-git-practice (feature)
$ git stash push -m "work on file1"
warning: in the working copy of 'file1.txt', LF will be replaced by CRLF the next time Git touches it
Saved working directory and index state On feature: Work on file1

asifs@Asif_Shaik MINGW64 ~/devops-git-practice (feature)
$ git status
On branch feature
Your branch is up to date with 'origin/feature'.

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    passwords.txt

nothing added to commit but untracked files present (use "git add" to track)

asifs@Asif_Shaik MINGW64 ~/devops-git-practice (feature)
$ touch file2.txt

asifs@Asif_Shaik MINGW64 ~/devops-git-practice (feature)
$ vi file2.txt

asifs@Asif_Shaik MINGW64 ~/devops-git-practice (feature)
$ git status
On branch feature
Your branch is up to date with 'origin/feature'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
    modified:   file2.txt

```

```

asifs@Asif_Shaik MINGW64 ~/devops-git-practice (feature)
$ git stash list
stash@{0}: On feature: Work on file1

asifs@Asif_Shaik MINGW64 ~/devops-git-practice (feature)
$ git stash pop
On branch feature
Your branch is up to date with 'origin/feature'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
    modified:   file1.txt
    modified:   file2.txt

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    passwords.txt

no changes added to commit (use "git add" and/or "git commit -a")
Dropped refs/stash@{0} (6413791079c3da759031688c483569fdbd5f0cc9)

```

Conclusion

The changes to file1.txt were temporarily stored using git stash, allowing you to work on file2.txt without committing the first file. The stashed work can later be restored using git stash pop.

16. Undo the stash file and start working on that again.

Objective

To restore the changes that were temporarily stored in the Git stash and continue working on the file.

Purpose

git stash pop is used to **restore stashed changes** back into the working directory and remove them from the stash list.

Implementation / Steps

Step 1: Check the stash

```
git stash list
```

Step 2: Restore the stashed changes

```
git stash pop
```

Step 3: Verify the file

```
git status
```

Step 4: Start working on the file again

Open the file:

```
vi file1.txt
```

```
asifs@Asif-Shaik MINGW64 ~/devops-git-practice (feature)
$ git stash pop
On branch feature
Your branch is up to date with 'origin/feature'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   file1.txt
        modified:   file2.txt

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        passwords.txt

no changes added to commit (use "git add" and/or "git commit -a")
Dropped refs/stash@{0} (6413791079c3da759031688c483569fdbd5f0cc9)
```

```

asifs@Asif_Shaik MINGW64 ~/devops-git-practice (feature)
$ vi file1.txt

asifs@Asif_Shaik MINGW64 ~/devops-git-practice (feature)
$ git status
On branch feature
Your branch is up to date with 'origin/feature'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   file1.txt
        modified:   file2.txt

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        passwords.txt

no changes added to commit (use "git add" and/or "git commit -a")

```

Conclusion

The stashed changes were successfully **restored using git stash pop**, and file1.txt is now available for further work. The stash entry is removed after a successful pop.

17. Generate a ssh-keygen and configure into github.

Objective

To generate an SSH key pair on the local system and configure the public key with GitHub so Git can authenticate with GitHub without entering a username and password for every operation.

Purpose

SSH authentication provides a secure way to connect your local Git installation with GitHub.

Local Computer

|

| SSH Key

↓

GitHub

- Private key → stays on your computer. Never share it.
- Public key → added to your GitHub account.
- SSH allows you to use Git operations such as clone, pull, and push securely.

Implementation / Steps

Step 1: Check whether an SSH key already exists

In Git Bash:

```
ls -la ~/.ssh
```

Step 2: Generate an SSH key

Use:

```
ssh-keygen -t ed25519 -C your-github-email@example.com
```

Step 3: Start the SSH agent

```
eval "$(ssh-agent -s)"
```

Step 4: Add your private key to the SSH agent

```
ssh-add ~/.ssh/id_ed25519
```

Step 5: Display your public key

```
cat ~/.ssh/id_ed25519.pub
```

Step 6: Add the key to GitHub

Open [GitHub SSH settings](#)

Then:

1. Click New SSH key
2. Title: My Git Bash PC
3. Key type: Authentication Key
4. Paste your copied public key into Key
5. Click Add SSH key

Step 7: Test the SSH connection

Back in Git Bash:

ssh -T git@github.com

Step 8: Change your repository remote from HTTPS to SSH

Your repository currently uses an HTTPS URL. Check it:

```
git remote -v
```

Change it to SSH:

```
git remote set-url origin git@github.com:asifshaik6558/devops-git-practice.git
```

Verify:

```
git remote -v
```

Step 9: Test Git push

```
git push origin feature
```

```
asif@asif-shaik MINGW64 ~/devops-git-practice (feature)
$ ssh-add ~/.ssh/id_rsa
Identity added: /c:/Users/asif/.ssh/id_rsa (asif@asif-shaik)

asif@asif-shaik MINGW64 ~/devops-git-practice (feature)
$ ssh-add -l
3072 SHA256:P4tcutoSYDRMk+8Zv1JJZxtlspc9JW6zpxd+0FIHTUU asif@asif-shaik (RSA)

asif@asif-shaik MINGW64 ~/devops-git-practice (feature)
$ ssh -T git@github.com
Hi asifshaik6558! You've successfully authenticated, but GitHub does not provide shell access.

asif@asif-shaik MINGW64 ~/devops-git-practice (feature)
$ git remote -v
origin https://github.com/asifshaik6558/devops-git-practice.git (fetch)
origin https://github.com/asifshaik6558/devops-git-practice.git (push)

asif@asif-shaik MINGW64 ~/devops-git-practice (feature)
$ git remote set-url origin git@github.com:asifshaik6558/devops-git-practice.git

asif@asif-shaik MINGW64 ~/devops-git-practice (feature)
$ git remote -v
origin git@github.com:asifshaik6558/devops-git-practice.git (fetch)
origin git@github.com:asifshaik6558/devops-git-practice.git (push)

asif@asif-shaik MINGW64 ~/devops-git-practice (feature)
$ git push origin feature
Everything up-to-date

asif@asif-shaik MINGW64 ~/devops-git-practice (feature)
$ cat ~/.ssh/id_rsa.pub
ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAQGD1jK3x0TFj77foTy9jzxc1SSxkGLx101J1EnzsKVPcSv6B9+YDv4ndVzEbs8MPJhl7xclCzP8x1JZxcE17F2eTYWxJA2XunLMQamKYH6Yf8LsYCjxv0V0WjTTC2P7STilfwKJ/11htTbZBuWNRG62GzW19SCdf9fAmpU
acss8jTDuZd7-cmmez1gYX0D3C02f8FPpobys9Bw1Wt8K1gRn0BUPV5GzAkd06agnk4RUzm0dHR/sjDyeq0d8EYebvCs+028Nrx5vYhuTYWoolNFtrz2Mro17Gf1qsFw6G17Pp01We0yU+2T8d51vR/YbreJNcgxmf8IQcyC9xwStBDMh4XZN+cg4P0Z0bghDpKpBjh15/uu
mo1Tugv9Td5pPyays5B7y+4ch8qayBmwaw0n5W1qj81jzFN6Ckx0D5b5WkpuCJIIjyb/ekZjWlgrQdUPx7yN/qjvH8Xdn3tct1R6Q81b1Y5Mo1D0DTgLo8+4qCINqM= asif@asif-shaik
```

Conclusion

The SSH key was successfully configured with GitHub, and the feature branch was successfully verified using an SSH-based Git connection.

18. Configure webhooks to github.

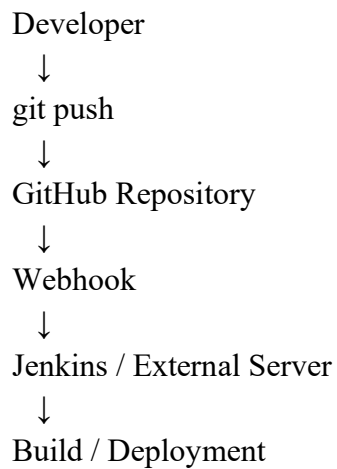
Objective

To configure a GitHub webhook so that GitHub automatically sends an HTTP request to an external service whenever a selected repository event occurs, such as a push. [GitHub Docs](#)

Purpose

A webhook is used to automatically notify another application when something happens in GitHub.

For example:



Implementation / Steps

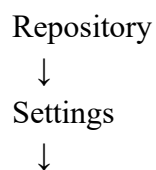
Step 1: Open your GitHub repository

Open your repository:

[devops-git-practice on GitHub](#)

Step 2: Open Webhooks

Go to:



Webhooks



Add webhook

Step 3: Enter the Payload URL

GitHub needs a publicly reachable URL that can receive the webhook.

For example, if you're connecting Jenkins:

http://YOUR_SERVER_IP:8080/github-webhook/

Step 4: Select Content type

Choose:

Application/json

Step 5: Add a Secret

Enter a strong random secret.

Example:

GitHubWebhook@2026#RandomSecret

Step 6: Select events

Under:

Which events would you like to trigger this webhook?

Step 7: Add the webhook

Click:

Add webhook

Step 8: Check webhook deliveries

Go back to:

Settings → Webhooks

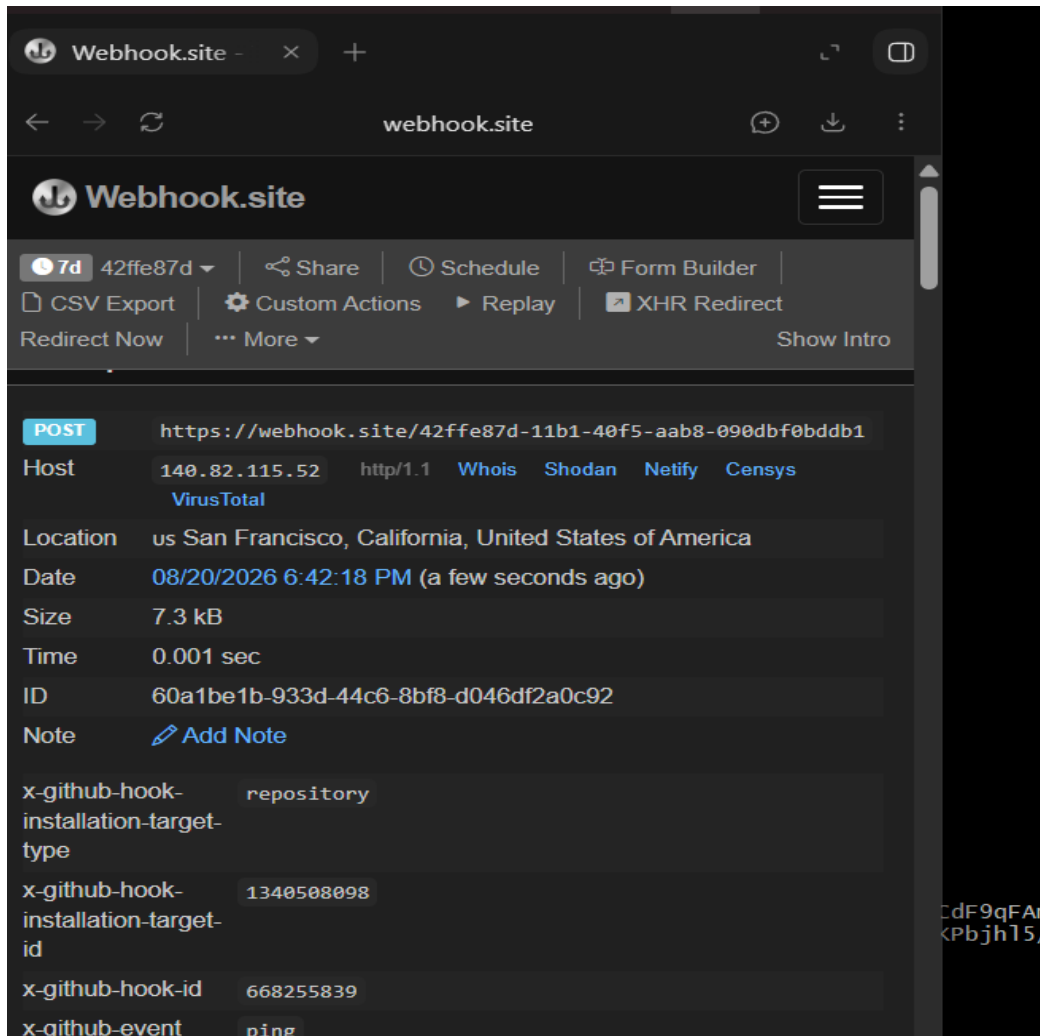
Step 9: Test with a Git push

From your local repository:

```
git add .
```

```
git commit -m "Test GitHub webhook"
```

```
git push origin feature
```



19. Basic understanding of .git file.

Purpose

The purpose of the `.git` directory is to store Git's internal information for a repository, such as commits, branches, staging information, configuration, and history.

Objective

To understand the basic structure and importance of the hidden `.git` directory and learn how Git uses it to manage a repository.

Implementation

Step 1: Go to your local repository

```
cd ~/devops-git-practice
```

Step 2: View hidden files

```
ls -la
```

Step 3: View `.git` contents

```
ls -la .git
```

Step 4: Check the current branch

```
cat .git/HEAD
```

Step 5: Check Git configuration

```
cat .git/config
```

```

asifs@Asif_Shaik MINGW64 ~/devops-git-practice (feature)
$ cd ~/devops-git-practice

asifs@Asif_Shaik MINGW64 ~/devops-git-practice (feature)
$ ls -la
total 29
drwxr-xr-x 1 asifs 197609  0 Aug 20 18:10 ./
drwxr-xr-x 1 asifs 197609  0 Aug 20 18:10 ../
drwxr-xr-x 1 asifs 197609  0 Aug 20 18:21 .git/
-rw-r--r-- 1 asifs 197609 20 Aug 20 17:28 .gitignore
-rw-r--r-- 1 asifs 197609 22 Aug 20 14:47 README.md
-rw-r--r-- 1 asifs 197609 35 Aug 20 18:10 file1.txt
-rw-r--r-- 1 asifs 197609 37 Aug 20 17:58 file2.txt
-rw-r--r-- 1 asifs 197609 32 Aug 20 17:20 passwords.txt

asifs@Asif_Shaik MINGW64 ~/devops-git-practice (feature)
$ ls -la .git
total 23
drwxr-xr-x 1 asifs 197609  0 Aug 20 18:21 ./
drwxr-xr-x 1 asifs 197609  0 Aug 20 18:10 ../
-rw-r--r-- 1 asifs 197609 41 Aug 20 17:59 AUTO_MERGE
-rw-r--r-- 1 asifs 197609 390 Aug 20 17:38 COMMIT_EDITMSG
-rw-r--r-- 1 asifs 197609 24 Aug 20 17:54 HEAD
-rw-r--r-- 1 asifs 197609 41 Aug 20 17:57 ORIG_HEAD
-rw-r--r-- 1 asifs 197609 375 Aug 20 18:21 config
-rw-r--r-- 1 asifs 197609 73 Aug 20 14:47 description
drwxr-xr-x 1 asifs 197609  0 Aug 20 14:47 hooks/
-rw-r--r-- 1 asifs 197609 342 Aug 20 17:59 index
drwxr-xr-x 1 asifs 197609  0 Aug 20 14:47 info/
drwxr-xr-x 1 asifs 197609  0 Aug 20 14:49 logs/
drwxr-xr-x 1 asifs 197609  0 Aug 20 17:57 objects/
drwxr-xr-x 1 asifs 197609  0 Aug 20 17:59 refs/

asifs@Asif_Shaik MINGW64 ~/devops-git-practice (feature)
$ cat .git/HEAD
ref: refs/heads/feature

asifs@Asif_Shaik MINGW64 ~/devops-git-practice (feature)
$ cat .git/config
[core]
    repositoryformatversion = 0
    filemode = false
    bare = false
    logallrefupdates = true
    symlinks = false
    ignorecase = true
[remote "origin"]
    url = git@github.com:asifshaik6558/devops-git-practice.git
    fetch = +refs/heads/*:refs/remotes/origin/*
[branch "main"]
    remote = origin
    merge = refs/heads/main
[branch "feature"]
    remote = origin
    merge = refs/heads/feature

```

Conclusion

The `.git` directory is the core internal directory of a Git repository. It stores information about commits, branches, configuration, and staging. Understanding `.git` helps us understand how Git tracks and manages changes in a project.

20. Check all the logs of git.

Purpose

To view the complete history of commits made in the Git repository, including commit IDs, authors, dates, and commit messages.

Objective

To understand and verify the changes and commits made in the local Git repository.

Implementation

Step 1: Go to your repository

```
cd ~/devops-git-practice
```

Step 2: View the complete commit history

```
git log
```

Step 3: View logs in one line

For a simpler view:

```
git log --oneline
```

Step 4: View all branches' logs

To see commits from all branches:

```
git log --all --oneline
```

For a graphical view:

```
git log --all --oneline --graph --decorate
```

Step 5: Exit the log screen

If Git opens the log in a scrolling screen, press:

Q

Useful Git Log Commands

Command	Purpose
git log	Shows detailed commit history
git log --oneline	Shows commits in one line
git log --all	Shows commits from all branches
git log --graph	Shows branch history graphically
git log --decorate	Shows branch/tag references
git log --all --oneline --graph --decorate	Shows a compact graphical history of all branches

```
asifs@Asif_Shaik MINGW64 ~/devops-git-practice (feature)
$ cd ~/devops-git-practice

asifs@Asif_Shaik MINGW64 ~/devops-git-practice (feature)
$ git log
commit 055b737a7a4fb9a9613b24876690b9621bc2fd40 (HEAD -> feature, origin/feature)
Author: Asif <asifshaik6558@gmail.com>
Date: Thu Aug 20 17:37:16 2026 +0530

    Revert "Add sample file"

    This reverts commit 584debcc4f2358a2648a983a22a0fd9ba7087595.

commit 584debcc4f2358a2648a983a22a0fd9ba7087595
Author: Asif <asifshaik6558@gmail.com>
Date: Thu Aug 20 15:18:53 2026 +0530

    Add sample file

commit d669c695aae35efa95bffb610c7deddd7a1a183d (origin/main, main)
Author: Asif <asifshaik6558@gmail.com>
Date: Thu Aug 20 15:13:50 2026 +0530

    Add two files

commit 0ffb7bf4396fd741829f73aca4620a62aea827916
Author: Asif <asifshaik6558@gmail.com>
Date: Thu Aug 20 14:49:27 2026 +0530

    Add README and gitignore

asifs@Asif_Shaik MINGW64 ~/devops-git-practice (feature)
$ git log --oneline
055b737 (HEAD -> feature, origin/feature) Revert "Add sample file"
584debc Add sample file
d669c69 (origin/main, main) Add two files
0ffb7f4 Add README and gitignore

asifs@Asif_Shaik MINGW64 ~/devops-git-practice (feature)
$ git log --all --oneline --graph --decorate
* 055b737 (HEAD -> feature, origin/feature) Revert "Add sample file"
* 584debc Add sample file
* d669c69 (origin/main, main) Add two files
* 0ffb7f4 Add README and gitignore
```

Conclusion

git log is used to check the history of commits in a Git repository. The command git log --all --oneline --graph --decorate is especially useful for seeing commits, branches, and their relationships in a simple format.

21.Rename the commit message.

Purpose

To change the message of the most recent Git commit to make it clear and meaningful.

Objective

To learn how to rename or modify a Git commit message using the git commit --amend command.

Implementation

1. Check the existing commit:

```
git log --oneline
```

. Rename the latest commit message:

```
git commit --amend -m "Updated commit message"
```

. Verify the changed commit message:

```
git log --oneline
```

```
asifs@Asif_Shaik MINGW64 ~/bash-project/my-bash-repo (main)
$ git log --oneline
6b7e563 (HEAD -> main) Add test bash script
37dca6b Update test script
e5e7861 (origin/shaikk, shaikk) Add feature script
ae86fae Add test bash script

asifs@Asif_Shaik MINGW64 ~/bash-project/my-bash-repo (main)
$ git commit --amend -m "Updated test bash script"
[main b587dc8] Updated test bash script
Date: Wed Aug 19 17:07:08 2026 +0530
1 file changed, 4 insertions(+)

asifs@Asif_Shaik MINGW64 ~/bash-project/my-bash-repo (main)
$ git log --oneline
b587dc8 (HEAD -> main) Updated test bash script
37dca6b Update test script
e5e7861 (origin/shaikk, shaikk) Add feature script
ae86fae Add test bash script

asifs@Asif_Shaik MINGW64 ~/bash-project/my-bash-repo (main)
$ git commit --amend -m "Updated test bash script"
[main 6bef3cb] Updated test bash script
Date: Wed Aug 19 17:07:08 2026 +0530
1 file changed, 4 insertions(+)

asifs@Asif_Shaik MINGW64 ~/bash-project/my-bash-repo (main)
$ git log --oneline
6bef3cb (HEAD -> main) Updated test bash script
37dca6b Update test script
e5e7861 (origin/shaikk, shaikk) Add feature script
ae86fae Add test bash script
```

Conclusion

The latest Git commit message was successfully renamed using `git commit --amend -m`. This helps maintain clear and meaningful commit history.

22.Merge multiple commits into single commit.

Purpose

To combine multiple Git commits into a single commit, making the Git history cleaner and easier to understand.

Objective

To learn how to use interactive rebase (`git rebase -i`) to merge multiple commits into one commit.

Implementation

Step 1: Check the current history

```
git log --oneline
```

Step 2: Start interactive rebase

To combine the latest 2 commits, run:

```
git rebase -i HEAD~2
```

Step 3: Change the second pick to squash

Change it to:

```
pick 37dca6b Update test script
squash 6bef3cb Updated test bash script
```

Step 4: Edit the final commit message

Git may open another editor showing the two commit messages.

Replace them with one message, for example:

Combined test script updates

Step 5: Verify

Run:

git log --oneline

```
MINGW64:/c/Users/asifs/bash-project/my-bash-repo
pick 37dca6b Update test script
pick 6bef3cb Updated test bash script

# Rebase e5e7861..6bef3cb onto e5e7861 (2 commands)
#
# Commands:
# p, pick <commit> = use commit
# r, reword <commit> = use commit, but edit the commit message
# e, edit <commit> = use commit, but stop for amending
# s, squash <commit> = use commit, but meld into previous commit
# f, fixup [-C | -c] <commit> = like "squash" but keep only the previous
#                               commit's log message, unless -C is used, in which case
#                               keep only this commit's message; -c is same as -C but
#                               opens the editor
# x, exec <command> = run command (the rest of the line) using shell
# b, break = stop here (continue rebase later with 'git rebase --continue')
# d, drop <commit> = remove commit
# l, label <label> = label current HEAD with a name
# t, reset <label> = reset HEAD to a label
# m, merge [-C <commit> | -c <commit>] <label> [# <oneline>]
# .       create a merge commit using the original merge commit's
# .       message (or the oneline, if no original merge commit was
# .       specified); use -c <commit> to reword the commit message
#
# These lines can be re-ordered; they are executed from top to bottom.
#
# If you remove a line here THAT COMMIT WILL BE LOST.
#
# However, if you remove everything, the rebase will be aborted.
#

MINGW64:/c/Users/asifs/bash-project/my-bash-repo
pick 37dca6b Update test script
squash| 6bef3cb Updated test bash script

# Rebase e5e7861..6bef3cb onto e5e7861 (2 commands)
#
# Commands:
# p, pick <commit> = use commit
# r, reword <commit> = use commit, but edit the commit message
# e, edit <commit> = use commit, but stop for amending
# s, squash <commit> = use commit, but meld into previous commit
# f, fixup [-C | -c] <commit> = like "squash" but keep only the previous
#                               commit's log message, unless -C is used, in which case
#                               keep only this commit's message; -c is same as -C but
#                               opens the editor
# x, exec <command> = run command (the rest of the line) using shell
# b, break = stop here (continue rebase later with 'git rebase --continue')
# d, drop <commit> = remove commit
# l, label <label> = label current HEAD with a name
# t, reset <label> = reset HEAD to a label
# m, merge [-C <commit> | -c <commit>] <label> [# <oneline>]
# .       create a merge commit using the original merge commit's
# .       message (or the oneline, if no original merge commit was
# .       specified); use -c <commit> to reword the commit message
#
# These lines can be re-ordered; they are executed from top to bottom.
#
# If you remove a line here THAT COMMIT WILL BE LOST.
#
# However, if you remove everything, the rebase will be aborted.
#
```

```
asifs@Asif_Shaik MINGW64 ~/bash-project/my-bash-repo (main)
$ git log --oneline
6bef3cb (HEAD -> main) Updated test bash script
37dca6b Update test script
e5e7861 (origin/shaikk, shaikk) Add feature script
ae86fae Add test bash script

asifs@Asif_Shaik MINGW64 ~/bash-project/my-bash-repo (main)
$ git rebase -i HEAD~2
[detached HEAD b6c7a43] Combined test script updates
Date: Wed Aug 19 16:52:54 2026 +0530
1 file changed, 5 insertions(+)
Successfully rebased and updated refs/heads/main.

asifs@Asif_Shaik MINGW64 ~/bash-project/my-bash-repo (main)
$ git log --oneline
b6c7a43 (HEAD -> main) Combined test script updates
e5e7861 (origin/shaikk, shaikk) Add feature script
ae86fae Add test bash script
```

Conclusion

Multiple Git commits were successfully combined into a single commit using interactive rebase. This helps maintain a clean and organized Git commit history.